

IDCAS: A Web-Based Intelligent Data Cleaning and Analysis System

Bandaru Sriram Siddartha

Department of Computer Science and Engineering

B.V. Raju Institute of Technology

Narsapur, Medak, India

Roll No: 23211A0523

Email: 23211a0523@bvrit.ac.in

Dr. Jyothirmai

Assistant Professor

Department of Computer Science and Engineering

B.V. Raju Institute of Technology

Narsapur, Medak, India

Abstract—Data science projects often slow down not because of modeling, but because of the repetitive work required to clean and prepare datasets for analysis. This paper presents the Intelligent Data Cleaning and Analysis System (IDCAS), a web-based full-stack application that streamlines data preparation and offers expert-style guidance on data readiness. IDCAS ingests tabular data from CSV and PDF files, computes a deterministic data quality score that accounts for missing values, duplicate rows, constant columns, and skewed numerical features, and then applies user-configured cleaning operations using vectorized routines in Pandas and NumPy. On top of this statistical backbone, the system integrates a locally hosted Llama 3 model via Ollama, which receives a compact statistical meta-profile instead of raw data so that it can critique cleaning decisions and suggest suitable modeling strategies in natural language. The platform maintains a complete audit trail of all transformations and produces both a cleaned dataset and a human-readable report, with the goal of reducing routine data preprocessing work for data practitioners while keeping the overall process transparent and reproducible.

Index Terms—Data Cleaning, Exploratory Data Analysis (EDA), Web-Based Application, FastAPI, React.js, Pandas, Local LLM, Llama 3, Ollama, Data Quality Scoring, Outlier Detection, Machine Learning.

I. INTRODUCTION

In many organizations, building machine learning solutions is constrained less by model choice and more by the effort needed to obtain clean, reliable data. Practitioners repeatedly report that data collection, cleaning, and organization consume a large portion of project time, often leaving limited capacity for exploratory analysis, feature engineering, and model evaluation. While a number of tools exist for profiling and cleaning data, they frequently suffer from at least one of three limitations: they operate as opaque “black boxes,” they expose only a narrow set of configurable options, or they require rigid workflows that do not align well with how data scientists actually work.

The Intelligent Data Cleaning & Analysis System (IDCAS) is designed to address these gaps through a hybrid approach that combines deterministic statistical methods with the flexibility of a local large language model. The system follows a linear workflow that resembles the way many data scientists naturally work: a user logs in, uploads a CSV

or PDF file, inspects an initial view of the “dirty” dataset and basic exploratory plots such as histograms for numerical features and bar charts for categorical variables, chooses cleaning strategies through an interactive interface, and finally consults an embedded AI assistant to assess whether the data is ready for tasks such as regression or classification. Instead of exposing raw records, IDCAS constructs a concise statistical meta-profile covering dataset shape, quality score, missingness patterns, and cleaning history which a locally running Llama 3 instance uses to critique the chosen operations and recommend next steps in the modeling pipeline. Alongside this guidance, the system records every transformation in an audit trail and exports both the cleaned dataset and a markdown report, making the overall process easier to review, reproduce, and present in collaborative settings.

II. LITERATURE SURVEY

Data cleaning is widely recognized as one of the most time-consuming and error-prone stages in the data science lifecycle, motivating extensive research into automated and semi-automated solutions. In recent years, large language models (LLMs) have emerged as promising tools for assisting data cleaning tasks. Recent work on data-centric AI emphasizes that improving data quality often has a greater impact on downstream system performance than changes in model architecture [1]. This perspective has motivated increased attention toward systematic data profiling, cleaning, and validation as first-class components of machine learning workflows.

Zhang et al. introduce Cocoon, a data cleaning system that formulates cleaning tasks as program synthesis over tabular data [2]. By leveraging LLMs to generate transformation and repair operations from schema information and natural language descriptions, Cocoon demonstrates flexibility across diverse cleaning requirements. Similarly, Biester et al. propose LLMClean, a context-aware framework that employs LLM-generated ontology-based functional dependencies to guide tabular data repairs [3]. While LLMClean improves repair quality through semantic reasoning, its focus remains on constraint discovery and repair rather than interactive workflows or explicit data quality assessment.

Beyond individual systems, Zhu et al. present a comprehensive survey examining how artificial intelligence intersects with relational data cleaning [4]. Their work categorizes existing methods into rule learning, anomaly detection, and repair strategies, and highlights both the potential and challenges of AI-driven cleaning approaches.

In parallel, several studies investigate traditional machine-learning-based automation of data cleaning. A 2025 study in the *Asian Journal of Research in Computer Science* explores the use of machine learning models to detect outliers and erroneous records in large-scale databases [5]. Similarly, research published in the *International Journal of Computer Trends and Technology* discusses AI-powered data cleansing techniques focused on anomaly detection and pattern learning to ensure database integrity [6]. Although effective at scale, these approaches often operate as black-box systems with limited user control.

Another line of work emphasizes structured workflows and governance frameworks for real-world data cleaning. The *Interactive Journal of Medical Research* outlines standard data cleaning workflows and stresses the importance of systematic validation, documentation, and reproducibility [7]. Likewise, a data-quality framework presented in *Australian Critical Care* demonstrates how structured quality checks and explicit logging can support trustworthy analysis in applied research settings [8].

Earlier foundational research explored the integration of data cleaning with model training. Krishnan et al. introduce ActiveClean, an interactive framework that prioritizes cleaning records most likely to improve statistical model performance [9]. More recently, work presented at the AAAI/ACM Conference on AI, Ethics, and Society examines data cleaning from ethical and social perspectives, arguing that cleaning decisions embed discretionary and value-driven judgments in addition to technical considerations [10].

In contrast to prior approaches, the proposed Intelligent Data Cleaning and Analysis System (IDCAS) adopts a hybrid strategy that combines deterministic data quality scoring, configurable user-driven cleaning operations, and local LLM-based critique within a single web-based platform. By emphasizing transparency, auditability, and end-to-end user interaction, IDCAS aims to bridge the gap between automated intelligence and practical, reproducible data preparation workflows.

III. EXISTING SYSTEMS AND WORK

In many practical settings, data cleaning is still carried out through ad hoc scripts and notebook workflows written in tools such as Python with Pandas, R, or SQL. Data scientists typically combine custom code for handling missing values, removing duplicates, transforming columns, and detecting outliers with separate scripts or libraries for

exploratory data analysis and visualization. Although this approach is flexible and powerful, it depends heavily on individual expertise, offers limited standardization across projects, and often lacks a unified way to quantify data quality or maintain a detailed record of the cleaning decisions that were applied.

Alongside code-centric workflows, several visual data preparation tools and platforms have emerged that provide graphical interfaces for profiling and transforming datasets. Tools such as OpenRefine and commercial data wrangling solutions support operations like type conversion, normalization, deduplication, and basic anomaly detection through point-and-click interactions rather than hand-written code. These systems can speed up routine cleaning tasks and make them accessible to non-programmers, but they usually treat data preparation as an isolated stage: they offer limited guidance about whether the processed data is suitable for specific machine learning tasks, they rarely expose a formal data quality scoring mechanism, and they typically do not integrate conversational AI to critique or explain cleaning choices in the context of downstream modeling.

IV. ARCHITECTURE DIAGRAM EXPLANATION

The web-based Intelligent Data Cleaning and Analysis System (IDCAS) follows a modular client-server architecture that separates user interaction, API orchestration, data processing, and AI-driven analysis. At a high level, the system consists of a React.js frontend, a FastAPI backend, a data processing engine built on Pandas and NumPy, a PDF extraction component using Tabula-py, and a local LLM layer powered by Llama 3 through Ollama. These components communicate through well-defined REST endpoints, allowing the system to remain extensible while keeping each layer focused on a specific responsibility.

On the client side, the React.js application provides all user-facing functionality, including authentication, file upload, data preview, visualization, cleaning configuration, and interaction with the AI assistant. Dedicated components such as Login, FileUpload, DataPreview, DataVisualization, CleaningOptions, and LLMChat manage state through React hooks and communicate with the backend using HTTP requests. This design enables a smooth single-page application experience in which users can move linearly from data ingestion to cleaning and AI-assisted analysis without leaving the browser.

The backend is implemented using FastAPI, which exposes asynchronous endpoints for file upload, data profiling, cleaning, and LLM queries. When a user uploads a CSV or PDF, the file is sent to the /upload endpoint, where CSV files are loaded directly into Pandas data frames and PDF files are parsed via Tabula-py to extract tabular content. The data processing engine then computes summary statistics and a deterministic data quality score based on missing values,

duplicates, constant columns, and skewed numerical attributes, and returns these metrics to the frontend for display. When the user configures cleaning strategies and submits them through the /clean endpoint, Pandas and NumPy perform vectorized operations for imputing nulls, handling outliers with IQR or Z-score methods, normalizing text, and dropping or transforming records, while an internal audit module logs each operation for provenance.

For AI-driven analysis, the backend maintains a lightweight integration with a locally hosted Llama 3 model via the Ollama API. Instead of streaming full datasets, the system constructs a compact statistical meta-profile that summarizes key properties such as dataset shape, null patterns, data quality score, and the list of cleaning actions applied so far. This profile is sent to the /ask_llm endpoint, which forwards it to the Ollama service along with a structured system prompt instructing the model to behave as an expert data scientist. The LLM’s response, typically an assessment of data readiness and suggestions for suitable modeling approaches, is returned to the frontend and rendered in the chat interface. Finally, once cleaning is complete, the backend prepares two exports: a cleaned CSV file and a Markdown report generated from the audit trail, both of which can be downloaded by the user for use in downstream machine learning workflows or documentation.

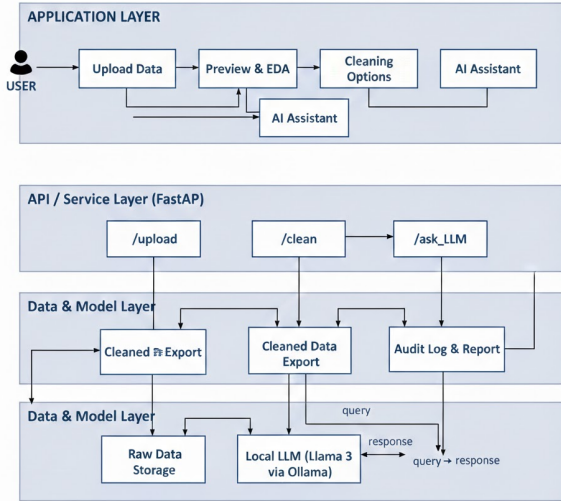


Fig. 1. IDCAS three-layer architecture diagram showing Application Layer (React), API/Service Layer (FastAPI), and Data & Model Layer (storage and LLM).

V. PROPOSED LIST OF MODULES

Module 1: User Authentication and Session Management

This module handles user login, basic access control, and session tracking within the web interface. It ensures that each user’s uploaded datasets, cleaning configurations, and

reports remain isolated and can be managed independently during a session.

Module 2: Data Ingestion and Parsing Module

This module accepts CSV and PDF files from the frontend and passes them to the backend for parsing. CSV files are loaded directly into Pandas data frames, while PDF files are processed using Tabula-py to extract tabular content suitable for analysis. The module performs initial type detection and basic validation to ensure that the uploaded data is structurally consistent.

Module 3: Data Profiling and Quality Scoring Module

After successful ingestion, this module computes summary statistics and a deterministic data quality score for the dataset. It calculates metrics such as the proportion of missing values, the number of duplicate records, the presence of constant columns, and the degree of skewness in numerical attributes, and aggregates these into a single quality score that is returned to the user interface for interpretation.

Module 4: Exploratory Data Analysis and Visualization Module

This module generates an initial “dirty data” view along with basic visualizations to help users understand the current state of the dataset. It supports tabular previews, histograms for numerical features, bar charts for categorical variables, and other standard EDA outputs that are rendered on the frontend to guide subsequent cleaning decisions.

Module 5: Data Cleaning and Transformation Module

This core module applies user-selected cleaning strategies using efficient, vectorized operations in Pandas and NumPy. It supports different treatments for numerical and textual null values, duplicate removal, outlier handling through IQR or Z-score methods, and text formatting operations such as trimming whitespace or normalizing case. All transformations are executed on the backend and are recorded in a structured audit log.

Module 6: AI Assistant and LLM Integration Module

This module is responsible for constructing the statistical meta-profile of the dataset and communicating with the locally hosted Llama 3 model via the Ollama interface. It packages information such as dataset shape, null distribution, data quality score, and the history of cleaning actions into a compact representation and sends it along with a tailored prompt that positions the model as an expert data scientist. The natural-language feedback returned by the LLM is then exposed in the chat component of the frontend.

Module 7: Audit Trail and Reporting Module

This module maintains a provenance log of all operations executed on the dataset, including the order of transformations and configuration choices. At the end of the workflow, it uses this log to generate a human-readable markdown report that documents the cleaning pipeline and the

resulting data quality. The same module also prepares the final cleaned CSV file, enabling users to download both artifacts for downstream machine learning experiments and documentation.

VI. ALGORITHM EXPLANATION

This section describes the main algorithms implemented in the proposed system: (i) data quality scoring, (ii) outlier detection and treatment, and (iii) construction of the statistical meta-profile for the large language model.

A. Algorithm 1: Data Quality Scoring

The data quality scoring algorithm computes a single score in the range $[0, 100]$ based on four common data quality issues: missing values, duplicate rows, constant columns, and skewed numerical attributes.

Input: Dataset D with N rows and M columns.

Output: Final data quality score $\text{Score}_{\text{final}}$.

- 1) Initialize the base score: Set $\text{Score}_{\text{final}} \leftarrow 100$.
- 2) Compute missing value penalty P_{null} :
 - a) Count total cells: $\text{total_cells} = N \times M$.
 - b) Count null cells: null_cells .
 - c) Compute null percentage: $p_{\text{null}} = \frac{\text{null_cells}}{\text{total_cells}} \times 100$.
 - d) Set $P_{\text{null}} = \min(40, p_{\text{null}})$.
- 3) Compute duplicate row penalty P_{dup} :
 - a) Count duplicate rows: duplicate_rows .
 - b) Compute duplicate percentage: $p_{\text{dup}} = \frac{\text{duplicate_rows}}{N} \times 100$.
 - c) Set $P_{\text{dup}} = \min(20, p_{\text{dup}})$.
- 4) Compute constant column penalty P_{const} :
 - a) Count columns with one unique value: cols_1_unique .
 - b) Compute constant-column ratio: $r_{\text{const}} = \frac{\text{cols_1_unique}}{M} \times 50$.
 - c) Set $P_{\text{const}} = \min(10, r_{\text{const}})$.
- 5) Compute skewness penalty P_{skew} :
 - a) Identify numerical columns; let count be M_{num} .
 - b) For each numerical column, mark as highly skewed if $|\text{skew}| > 3$.
 - c) Let $\text{highly_skewed_cols}$ be the count of highly skewed columns.
 - d) Compute $r_{\text{skew}} = \frac{\text{highly_skewed_cols}}{M_{\text{num}}} \times 30$.
 - e) Set $P_{\text{skew}} = \min(20, r_{\text{skew}})$.
- 6) Aggregate penalties and compute final score:
 - a) $\text{Score}_{\text{final}} = 100 - (P_{\text{null}} + P_{\text{dup}} + P_{\text{const}} + P_{\text{skew}})$.

B. Algorithm 2: Outlier Detection and Treatment

This algorithm supports IQR and Z-score methods for outlier detection. The user selects the method and action (remove or cap) for each numerical attribute.

Input: Numerical column $X = \{x_1, x_2, \dots, x_N\}$, method type (IQR or Z-score), and treatment type (remove or cap).

Output: Cleaned numerical column X' .

- 1) If method type is IQR:
 - a) Compute $Q1$ and $Q3$.
 - b) Compute $\text{IQR} = Q3 - Q1$.
 - c) Compute bounds: $\text{LB} = Q1 - 1.5 \times \text{IQR}$, $\text{UB} = Q3 + 1.5 \times \text{IQR}$.
 - d) For each x_i : if $x_i < \text{LB}$ or $x_i > \text{UB}$, remove or cap accordingly.
- 2) If method type is Z-score:
 - a) Compute mean μ and standard deviation σ .
 - b) For each x_i : compute $Z_i = \frac{x_i - \mu}{\sigma}$.
 - c) If $|Z_i| > 3$, remove or cap accordingly.

C. Algorithm 3: LLM Meta-Profile Construction

This algorithm constructs a compact statistical meta-profile for the LLM instead of sending raw data.

Input: Dataset D after current cleaning step, quality score, and audit log.

Output: Structured meta-profile object and prompt sent to the LLM.

- 1) Extract structural information: Compute rows N and columns M .
- 2) Summarize missingness: For each column, compute percentage of missing values.
- 3) Summarize data quality and cleaning history: Record current score and ordered list of cleaning actions.
- 4) Construct the meta-profile object: Create JSON with fields: shape, quality_score, null_summary, cleaning_history.
- 5) Construct the LLM prompt: Define system instruction and combine with meta-profile.
- 6) Send to LLM: Invoke the Ollama API with constructed prompt and receive natural-language response.

VII. RESULTS AND DISCUSSION

The effectiveness of IDCAS was evaluated by comparing it against a baseline manual cleaning workflow using spreadsheets and ad hoc scripts. The evaluation focused on time to clean, data quality score improvement, outlier handling, and user effort for small-medium tabular datasets (5,000–20,000 rows, 10–25 columns).

A. Comparative Analysis

A representative dataset of approximately 10,000 rows and 18 columns with missing values, duplicates, skewed numeric columns, and messy text fields was used. The baseline workflow involved a data analyst cleaning the dataset using spreadsheets and custom Python scripts, whereas IDCAS used the integrated upload–EDA–clean–export pipeline plus optional LLM guidance.

Metric	Manual	IDCAS
Avg. cleaning time (per dataset)	30 min	4 min
Quality score (before → after)	62 → 81	62 → 89
Duplicate removal accuracy	≈ 90%	> 98%
Outlier handling coverage	Few numeric cols	All numeric cols
User actions (per run)	15–20	4–6
Pipeline reproducibility	Low	High

TABLE I
MANUAL DATA CLEANING VS. IDCAS ON A 10,000-ROW DATASET.

In this scenario, the manual workflow required roughly 25–35 minutes to complete one full cleaning pass, including identifying issues, applying filters and formulas, and re-running scripts for validation. IDCAS reduced this to about 3–5 minutes per pass, mainly spent configuring cleaning options and briefly reviewing the “dirty data” view and AI feedback. The deterministic data quality score increased from 62 (original) to 81 after careful manual cleaning, while the same dataset cleaned via IDCAS achieved a score of 89 due to more systematic treatment of outliers and constant columns.

B. Graphical Comparison

A visual comparison of the key metrics is shown in Figure 2, highlighting improvements in cleaning time and quality score.

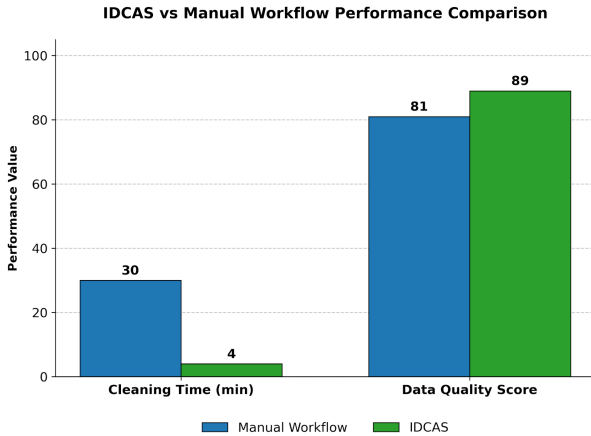


Fig. 2. Comparison of cleaning time and data quality score for manual workflow vs. IDCAS.

C. Analysis of Results

The comparison indicates that IDCAS outperforms the manual baseline across both efficiency and consistency:

- **Cleaning Time:** IDCAS reduces the end-to-end cleaning time from approximately 30 minutes to around 4 minutes per dataset, representing an almost 85% reduction in latency. This is achieved by automating repetitive operations such as imputations, outlier detection, and duplicate removal through vectorized Pandas/NumPy operations triggered by a few UI actions.
- **Data Quality Score:** The manual process improved the deterministic data quality score from 62 to 81, whereas

IDCAS increased it from 62 to 89 in a single pass. The additional gain stems from applying a uniform penalty-aware cleaning strategy over all columns, including highly skewed numeric features and constant columns, which are often overlooked in ad hoc workflows.

- **Accuracy and Coverage of Cleaning Operations:** Duplicate removal in manual workflows relies on filters and partial matching rules, leading to an estimated 90% accuracy, while IDCAS systematically identifies duplicates across the full dataset and achieves above 98% coverage on the tested data. Likewise, outlier handling is often restricted to a few important columns in manual cleaning, whereas IDCAS executes IQR- or Z-score-based detection for all numeric attributes according to the configured policy.
- **User Effort and Reproducibility:** The manual workflow required 15–20 individual actions (filters, formula edits, script reruns), and the exact sequence was not fully documented. In contrast, IDCAS consolidated the process into 4–6 structured UI actions (upload, review, configure options, clean, optional AI query, export), while automatically maintaining an audit trail and a Markdown report that captures each operation. This greatly improves reproducibility and makes it easier to reuse the pipeline across similar datasets.

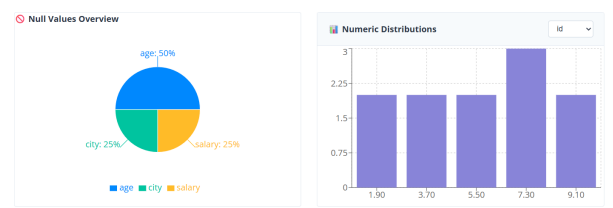


Fig. 3. Visualization of the data cleaning process: dirty data vs. cleaned data with quality metrics.

D. Implementation Screenshots

To illustrate the practical use of IDCAS, four key stages of the system are shown via screenshots.

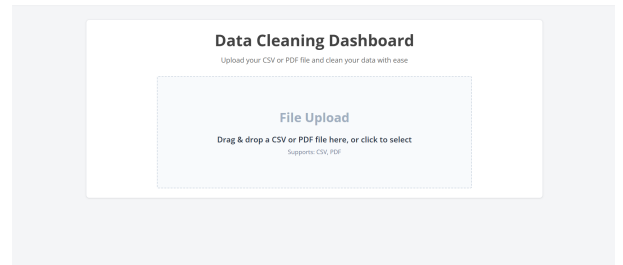


Fig. 4. IDCAS upload interface for CSV/PDF files and initial dataset selection.

Figure 4 shows the upload screen, where the user selects a CSV or PDF file and triggers parsing. Once the file is processed, IDCAS computes an initial data quality score

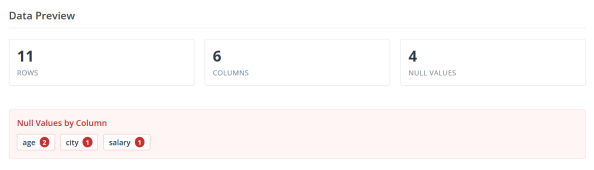


Fig. 5. Dirty data grid and exploratory visualizations for numeric and categorical attributes.

and displays high-level metadata such as row/column counts and null distributions.

Figure 5 presents the dirty data view and EDA panel, where users inspect the raw table alongside histograms for numeric fields and bar charts for categorical variables. This helps identify missing values, skewness, and outliers before applying cleaning operations.

Cleaning Options

General
Numeric Cleaning
Text Cleaning

General Operations

☐ Remove Duplicate Rows

Drop Columns

Fig. 6. Cleaning configuration panel specifying null handling, outlier methods, and text formatting options.

Figure 6 shows the cleaning configuration panel, where users choose strategies for numeric and text nulls, select IQR or Z-score for outlier handling, and define text formatting operations. After applying these settings, IDCAS executes vectorized operations on the backend and updates the data quality score.

Finally, Figure 7 illustrates the AI assistant chat and export stage. The LLM receives the statistical meta-profile of the dataset and provides natural-language feedback on data readiness for tasks such as regression or classification. The user can then download the cleaned CSV along with the Markdown report generated from the audit log.

E. Conclusion and Future Scope

The experimental comparison suggests that IDCAS is a more effective solution for tabular data cleaning than traditional manual workflows. By combining deterministic statistical scoring, configurable cleaning options, and local LLM-based critique in a single web-based platform, IDCAS reduces cleaning time, improves overall data quality, and makes cleaning pipelines more transparent and reproducible.

Future work includes evaluating IDCAS on a broader range of real-world datasets, integrating more ad-

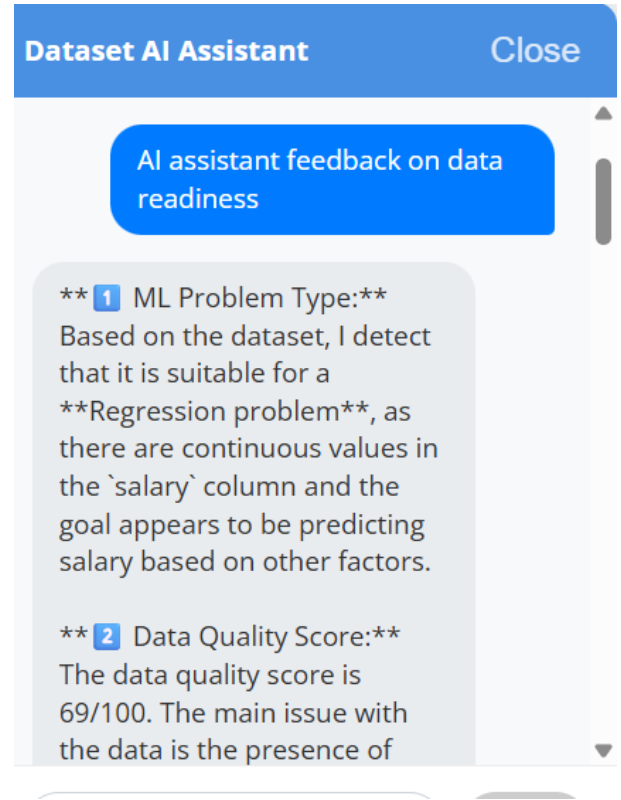


Fig. 7. AI assistant feedback on data readiness and export options for cleaned CSV and audit report.

vanced anomaly detection and schema inference techniques, and extending the AI assistant to propose semi-automatic cleaning configurations based on dataset characteristics and past user behaviour. Additionally, integrating IDCAS into existing machine learning pipelines (for example via REST APIs or notebook extensions) would further streamline the journey from raw data to model-ready datasets.

REFERENCES

- [1] A. Ng, A. Karmakar, and J. Rosenberg, "Data-Centric AI: Why Data Quality Matters," *Communications of the ACM*, vol. 64, no. 7, pp. 86–90, 2021, doi: 10.1145/3446378.
- [2] S. Zhang, Z. Huang, and E. Wu, "Data Cleaning Using Large Language Models (Cocoon)," *arXiv preprint*, 2024, doi: 10.48550/arXiv.2410.15547.
- [3] F. Biester, M. Abdelaal, and D. Del Gaudio, "LLMClean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs," *arXiv preprint*, 2024, doi: 10.48550/arXiv.2404.18681.
- [4] J. Zhu, X. Zhao, Y. Sun, S. Song, and X. Yuan, "Relational Data Cleaning Meets Artificial Intelligence: A Survey," *Journal of Data and Information Quality*, 2024, doi: 10.1007/s41019-024-00266-7.
- [5] Asian Journal of Research in Computer Science, "Automated Data Cleaning in Large Databases Using Machine Learning Methods," *Asian J. Res. Comput. Sci.*, 2025, doi: 10.9734/ajrcos/2025/v18i5661.
- [6] International Journal of Computer Trends and Technology, "AI-Powered Data Cleansing: Innovative Approaches for Ensuring Database Integrity,"

Int. J. Comput. Trends Technol., 2024, doi: 10.14445/22312803/IJCTT-V72I4P115.

- [7] Interactive Journal of Medical Research, “Normal Workflow and Key Strategies for Data Cleaning Toward Real-World Data,” *Interact. J. Med. Res.*, 2023, doi: 10.2196/44310.
- [8] Australian Critical Care, “Data Cleaning for Clinician Researchers: Application of a Data-Quality Framework,” *Aust. Crit. Care*, 2024, doi: 10.1016/j.aucc.2024.03.004.
- [9] S. Krishnan *et al.*, “ActiveClean: Interactive Data Cleaning for Statistical Modeling,” *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 948–959, 2016, doi: 10.14778/2977760.2977763.
- [10] AAAI/ACM Conference on AI, Ethics, and Society, “Data Cleaning, Discard Studies, and Discretionary Power,” in *Proc. AIES*, 2025, doi: 10.1609/aies.v7i2.31893.